It's Not the Capability: Harness Sensitivity Is Non-Monotone Across LLM Agent Tiers

Yong-eun Cho
KailosLab
Seoul, Republic of Korea
kevin@kailoslab.com

Abstract

A prevalent assumption in LLM agent deployment holds that more structured harnesses universally improve reliability, and that higher-capability models need proportionally less structural guidance—together implying a *monotone inverse* relationship between model capability tier and optimal harness complexity. We test this hypothesis through a controlled 432-run experiment crossing six models across four capability tiers with three harness conditions (*light*, *balanced*, *strict*) on HEAT-24, a 24-task synthetic benchmark with git-based workspace verification. Our results refute the monotone inverse relationship on two fronts. First, for the frontier *chat* model evaluated (Gemini 2.5 Flash), increased harness verbosity *lowers* VTSR by 29–38 percentage points—a **harness-complexity paradox**. Second, for the frontier *reasoning* model evaluated (Qwen3.5-122B, extended thinking enabled), strict harness achieves the *highest* VTSR (91.7%) and the *lowest* latency, the opposite of the prediction. Within the constrained tier, a 2 B model (Gemma4:e2B) matches strong-open-tier stability at 91.7% across all harnesses. Because each tier is represented by a single model in this study, these results should be interpreted as model-specific observations; harness sensitivity appears **non-monotone** across the models evaluated, and depends critically on model type (chat vs. reasoning). We introduce a six-label failure taxonomy showing that `format_violation` dominates capable-model failures while `wrong_file` dominates low-capability failures, and we derive practical tier-aware harness selection guidelines.

1 Introduction

Autonomous LLM agents that read, reason about, and modify workspace artifacts are increasingly deployed in software engineering (Liu et al., 2024; Jimenez et al., 2024), document processing, and operational workflows. The quality of the *harness*—the system-level prompt that specifies task scope, allowed operations, output format, and verification procedure—is widely believed to be a primary lever for improving agent reliability (Yao et al., 2023; Shinn et al., 2023).

Existing benchmarks evaluate agents on a fixed harness and report aggregate accuracy, obscuring the interaction between harness complexity and model capability. Practitioners routinely apply strict, highly-structured harnesses to all models in a deployment fleet under two implicit assumptions: that more structure always improves reliability, and that higher-capability models need less structural guidance—forming a *monotone inverse* relationship between capability tier and optimal harness complexity. We ask whether this monotone inverse hypothesis holds empirically across a diverse set of capability tiers, and whether the answer differs between chat-oriented and reasoning-oriented frontier models.

We make the following contributions:

- **HEAT-24** (**H**arness **E**valuation for **A**gent **T**asks), a deterministic 24-task synthetic benchmark with workspace-level git-based verification covering six task categories.
- The first controlled empirical test of the monotone inverse capability-harness hypothesis, crossing six models across four capability tiers with three harness conditions (432 total runs).
- Evidence that the hypothesis fails in opposite directions simultaneously: a **harness-complexity paradox** (strict harness *hurts* the frontier chat model) and a **non-monotonic pattern** (strict harness *helps* the frontier reasoning model most).
- The finding that parameter count is an unreliable proxy for harness sensitivity: a 2 B model (Gemma4:e2B) matches strong-open-tier sta-

bility, demonstrating that instruction-tuning quality is the true moderating variable.

- A six-label failure taxonomy and practical **tier-aware and type-aware harness selection** guidelines.

2 Related Work

LLM agent benchmarks. Liu et al. (2024) evaluate LLMs across eight interactive environments and show strong capability gaps between frontier and open-source models. Wei et al. (2022) demonstrate that chain-of-thought reasoning only reliably emerges above a model-size threshold, suggesting that structured prompts may impose undue cognitive overhead on smaller models—a pattern we observe in our constrained tier.

Instruction following and format compliance. Lou et al. (2024) survey instruction-following capabilities and find that compliance degrades as instruction complexity increases, directly motivating our harness-complexity investigation. Sclar et al. (2024) show that subtle changes in prompt formatting cause up to 76-point performance differences across open-source models, establishing that prompt structure is a major source of variance—a concern we directly investigate at the harness level. Mizrahi et al. (2024) argue that single-prompt evaluations are brittle and call for multi-prompt evaluation, supporting our cross-harness design. Work on structured output generation (Geng et al., 2025) shows that JSON and schema compliance varies substantially across model families, motivating our format-sensitive task category. Deng et al. (2025) find that separating task-solving from output formatting improves both dimensions, consistent with the format violations we observe when elaborate process instructions are mixed with output format requirements.

Agent scaffolding. ReAct (Yao et al., 2023) and Reflexion (Shinn et al., 2023) demonstrate that structured reasoning-action loops improve agent performance. Our study complements this by asking whether different *levels* of harness structure suit different model tiers, and whether extended thinking modes interact with harness structure in predictable ways.

Prompt complexity and performance inversion. Schulhoff et al. (2024) provide a systematic survey of prompting techniques and catalog conditions under which prompt engineering improves or degrades performance, providing a broad empirical context for our harness-complexity findings. Hakim (2026) find that brevity constraints on model outputs reverse performance hierarchies across model scales—larger models become relatively *worse* when forced to be concise. Khan (2025) argue that increasing prompt specificity can invert the expected performance ordering, with simpler prompts outperforming engineered ones for capable models. Both findings align with our harness-complexity paradox: the relationship between instruction richness and task success is non-monotonic and tier-dependent.

Self-correction and error recovery. Li (2025) decompose LLM self-correction and find an accuracy–correction paradox where stronger models make "deeper" errors that resist self-correction, while weaker models make more tractable surface errors—a pattern partially consistent with our failure taxonomy.

3 The HEAT-24 Benchmark

3.1 Workspace and Task Design

All tasks operate on a shared synthetic workspace containing twelve files: configuration YAML and JSON, Python source and test files, Markdown documentation, a CSV data file, and changelog fragments. The workspace is initialized as a git repository before each run; verification uses `git diff` to detect and scope file changes. All twelve workspace files are injected into every harness prompt, enabling models to read any file without external tool access.

We define 24 tasks across six categories (Table 1):

Category	# Tasks
`inspect_local`	4
`structured_edit`	4
`format_sensitive`	4
`verification_recovery`	4
`repair`	4
`multi_step_ops`	4

Table 1: HEAT-24 task categories (4 tasks each; 24 total). `inspect_local`: read files, return JSON; `structured_edit`: modify one file; `format_sensitive`: emit strict JSON schema; `verification_recovery`: fix bug, run tests; `repair`: correct malformed content; `multi_step_ops`: coordinate multiple files. Each task has a deterministic binary verifier.

Tasks are designed to have deterministic, binary outcomes. Verifiers check: JSON key presence and values, git-scoped file modifications, YAML/JSON parse validity, and substring presence. File modifications expressed in model output use a structured `<<<WRITE:path>>>` ··· `<<<END>>>` marker that the harness runner parses and applies to the workspace before verification. This marker format is included in the raw task instruction for all file-modification tasks across all three harness conditions; harness conditions differ only in the additional process instructions, scope constraints, and verification specifications layered on top.

3.2 Harness Conditions

We define three harness conditions of increasing structural complexity:

Light A two-line prompt: role statement plus the raw task instruction. No format specification, no scope constraint, no verification procedure.

Balanced Adds a four-step process template (plan, execute, check, respond) and lists the allowed files. No schema or verification spec.

Strict Adds six explicit stages (preflight / plan / execute / verify / recover / report), an allowed-file list, explicit success criteria, a verification specification, and instructions to express file changes using the `<<<WRITE:path>>>` marker.

3.3 Models

We evaluate six models spanning four capability tiers (Table 2). Tier assignments are based on deployment characteristics—parameter count and inference infrastructure—*not* on task performance, to avoid circularity. The goal is to test whether this conventional classification scheme predicts harness sensitivity. Each tier is represented by a single model in this study; tier-level claims should therefore be interpreted as model-specific observations pending replication with additional models within each tier. API-hosted models (Gemini 2.5 Flash, Qwen3.5-122B, GPT-OSS-120B) were queried with provider-default temperature and sampling settings at the time of the experiment; exact parameter values are logged in the released runner scripts. Ollama-hosted models used `think=False` to suppress chain-of-thought tokens, with context length fixed at 4096 tokens via per-model Modelfiles. Qwen3.5-122B's "Frontier-Reasoning" classification reflects extended thinking being enabled as an inference configuration choice across all runs, not an architectural property; results may differ if extended thinking were disabled.

3.4 Metrics and Failure Taxonomy

We report two metrics per run:

- **TSR** (Task Success Rate): binary pass/fail as determined by the workspace verifier.
- **VTSR** (Verified Task Success Rate): identical to TSR in this benchmark (all verifiers complete without infrastructure error); the distinction is maintained for deployments where verifier failures must be separated from model failures.

Failures are assigned one of six labels by an automated rule-based classifier that inspects git diff output, JSON parse results, and test execution logs; no manual labeling was performed: `format_violation` (output not parseable as required schema), `wrong_answer` (parseable but incorrect value), `wrong_file` (modified a file outside the allowed set), `missing_change` (no file modification detected), `unrelated_change` (modification to a correct file but wrong content), `tests_still_fail` (code change did not fix targeted test failure).

4 Results

4.1 Overall Performance by Harness Condition

Table 3 reports mean VTSR across 24 tasks for each model–harness combination (24 runs per cell). Because each tier is represented by a single model, cross-tier comparisons are exploratory; differences between tier rows reflect model-specific observations rather than tier-level laws.

4.2 Harness-Complexity Paradox: Frontier Chat Models

For Gemini 2.5 Flash (Frontier-Proprietary), light harness achieves VTSR = **95.8%**, dropping to 58.3% under balanced (-37.5 pp) and 66.7% under strict (-29.2 pp). The dominant failure mode in both complex conditions is `format_violation` (10 of 10 balanced failures; 8 of 8 strict failures).

Category-level analysis (Table 4) localizes the paradox to `inspect_local` and

Model	Tier	Params	Provider	Notes
Gemini 2.5 Flash	Frontier-Proprietary	—	Google AI Studio	Proprietary API
Qwen3.5-122B-A10B	Frontier-Reasoning	122B (10B act.)	vLLM (self-hosted)	MoE; extended thinking enabled (think-budget default)
GPT-OSS-120B	Strong-Open	120B	Groq Cloud	
Qwen3.5:2B	Constrained	2B	Ollama (local)	4096-token context Modelfile
LLaMA 3.2	Constrained	3B	Ollama (local)	4096-token context Modelfile
Gemma4:e2B	Constrained	2B	Ollama (local)	4096-token context Modelfile

Table 2: Models evaluated across four capability tiers. Tier assignments are based on deployment characteristics (parameter count and inference infrastructure), not on task performance. Model names for constrained-tier models reflect Ollama tags for the corresponding Google/Meta/Alibaba base models.

Model	Tier	Light	Balanced	Strict
Gemini 2.5 Flash	FP	**95.8**	58.3	66.7
Qwen3.5-122B-A10B	FR	87.5	75.0	**91.7**
GPT-OSS-120B	SO	**95.8**	**95.8**	87.5
Qwen3.5:2B	C	0.0	**58.3**	4.2
LLaMA 3.2	C	16.7	4.2	**20.8**
Gemma4:e2B	C	**91.7**	**91.7**	**91.7**

Table 3: Mean VTSR (%) by model and harness condition ($n = 24$ per cell, $k = 1$ repeat). **Bold** = best per row. Tier codes: FP = Frontier-Proprietary, FR = Frontier-Reasoning, SO = Strong-Open, C = Constrained. GPT-OSS-120B strict T13–T24 was re-evaluated using a second API key after the original run exhausted the 200 k TPD rate limit; the re-evaluation used identical prompt templates and provider-default parameters, completed within the same calendar week as the original run. Representative 95% Wilson CIs (for $n = 24$): 95.8% → [78.9, 99.9], 75.0% → [53.3, 90.2], 58.3% → [36.6, 78.2], 0% → [0.0, 14.2]. Results should be interpreted as preliminary evidence pending $k \geq 3$ repetitions for statistical reliability.

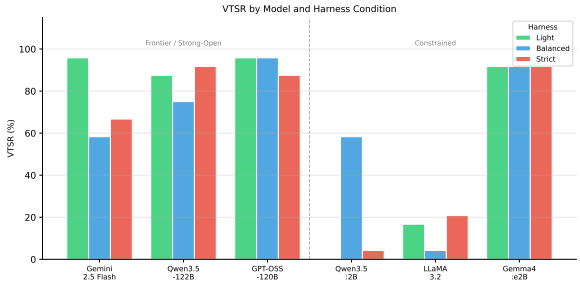

Figure 1: VTSR (%) by model and harness condition. The dashed line separates frontier/strong-open (left) from constrained (right) tiers.

`format_sensitive` tasks: Gemini achieves 100% on both categories under light harness but 0% on `inspect_local` and 25% on `format_sensitive` under strict. Tasks that require structured file editing (`structured_edit`, `repair`) remain at 100% across all harnesses, confirming that the paradox is specific to *format-sensitive output tasks*, not general capability regression.

We hypothesize that elaborate multi-stage instructions shift the generation distribution toward explanatory prose, causing the model to narrate its reasoning rather than emitting the required JSON schema directly. This aligns with the finding of Deng et al. (2025) that task-solving and output formatting compete as partially conflicting objectives.

4.3 Non-Monotonic Pattern: Frontier Reasoning Models

Qwen3.5-122B (Frontier-Reasoning, extended thinking enabled) exhibits a *non-monotonic* response to harness complexity: VTSR is lowest under balanced harness (75.0%), highest under strict (91.7%), and intermediate under light (87.5%). To our knowledge, this is the first empirical documentation of a tier-specific non-monotonic interaction between harness complexity and model type.

Notably, mean inference latency under strict harness (23.3 s) is *lower* than under light (35.4 s) or balanced (38.5 s), consistent with explicit constraints reducing the length of the model's thinking chains before output.

Category analysis reveals structure: balanced-harness failures concentrate in `inspect_local` (only 25% pass) and `format_sensitive` (50% pass), both of which recover to 100% under strict. This pattern is consistent with the reasoning model using the strict harness's explicit success criteria as scaffolding for its chain-of-thought, reducing ambiguity in what constitutes a correct output. Under the balanced harness, partial structure may create conflicting signals that the extended thinking process amplifies rather than resolves.

4.4 Strong-Open Model

GPT-OSS-120B (Strong-Open, via Groq) achieves equal and near-perfect performance under light and balanced harnesses (95.8% each), demonstrating robustness to harness complexity across those two conditions. Under strict harness, VTSR is 87.5% (21/24), with T14, T16 (`wrong_file`) and T24 (`format_violation`) as the three failures—a modest -8.3 pp gap relative to light and balanced. The initial strict run was contaminated by Groq's 200 k tokens-per-day (TPD) rate limit (T13–T24 returned empty outputs); the reported 87.5% figure is from a clean re-evaluation using a second API key. This pattern—light $\approx$ balanced $\gg$ strict, with strict still substantially above chance—distinguishes the Strong-Open tier from both frontier tiers: it does not suffer the full harness-complexity paradox (only -8.3 pp, not -29 pp), nor does it exhibit the non-monotonic pattern.

4.5 Constrained-Tier Models

The three constrained-tier models exhibit three distinct patterns that together reveal the heterogeneity within this tier.

Qwen3.5:2B—balanced-harness optimum. Qwen3.5:2B achieves 0% VTSR under the light harness, 58.3% under balanced, and only 4.2% under strict. The light-harness failures are dominated by `wrong_file` (15/24) and `format_violation` (9/24), indicating that without any structural guidance the model cannot reliably locate the target file or produce required output schemas. The balanced harness's moderate structure (four-step process template plus allowed-file list) provides sufficient scaffolding for this model to succeed on the majority of tasks. Strict harness then *reverses* the gain: the six-stage process template with verification specifications appears to exceed this model's instruction-following capacity, collapsing performance back toward zero. This **inverted-U pattern**—light $<$ strict $<$ balanced—stands in direct contrast to both the frontier chat model (light $>$ strict $>$ balanced) and the frontier reasoning model (strict $>$ light $>$ balanced), empirically demonstrating that no harness condition is universally optimal.

LLaMA 3.2—low capability, harness-insensitive. LLaMA 3.2 achieves uniformly low VTSR across all conditions (light: 16.7%, balanced: 4.2%, strict: 20.8%). Failures are dominated by `wrong_file` across all harnesses, with `format_violation` rising under balanced and strict. The near-flat performance curve ($\leq$21% in any condition) indicates that this model lacks the baseline instruction-following capability required to benefit from structural harness guidance. The slight strict advantage (20.8% vs. 16.7% light) is within noise and does not constitute a reliable pattern.

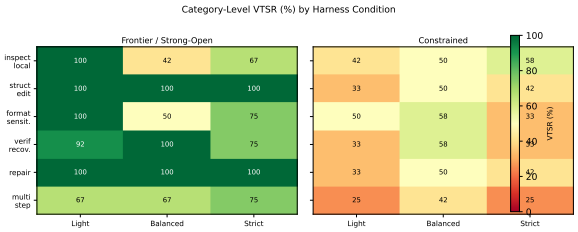

Figure 2: Category-level VTSR heatmaps for frontier/strong-open (left) and constrained (right) tiers. Green = high VTSR; red = low.

Gemma4:e2B—frontier-level stability. Gemma4:e2B achieves 91.7% VTSR under each of the three harness conditions, matching the stability profile of GPT-OSS-120B despite having approximately 60$\times$ fewer parameters. The two failures per condition span different failure types (`wrong_answer`, `format_violation`, `wrong_file`) rather than repeating the same label, indicating isolated variance rather than systematic harness-induced failure. This result challenges parameter count as a sufficient proxy for capability-tier classification in harness-sensitivity studies: Gemma4:e2B's instruction-tuning quality places its operational behavior firmly in the strong-open tier despite its 2 B parameter count. We discuss this finding further in §5.

4.6 Performance by Task Category

Table 4 reports macro-average VTSR by task category and harness condition, separately for the frontier/strong tier (Gemini, Qwen3.5-122B, GPT-OSS-120B) and the constrained tier (Qwen3.5:2B, LLaMA 3.2, Gemma4:e2B).

For frontier/strong models, `structured_edit` and `repair` achieve 100% across all harnesses, establishing them as reliable baselines unaffected by harness complexity. The `inspect_local` and `format_sensitive` categories show the strongest harness sensitivity, confirming that JSON-output tasks are the primary locus of format-violation failures. `multi_step_ops` is the only category where strict (75.0%) outperforms both light and

	Frontier/Strong			Constrained		
Category	**Li**	**Ba**	**St**	**Li**	**Ba**	**St**
`inspect_local`	**100.0**	41.7	66.7	41.7	50.0	**58.3**
`structured_edit`	**100.0**	**100.0**	**100.0**	33.3	**50.0**	41.7
`format_sensitive`	**100.0**	50.0	75.0	**50.0**	58.3	33.3
`verification_recovery`	91.7	**100.0**	75.0	33.3	**58.3**	33.3
`repair`	**100.0**	**100.0**	**100.0**	33.3	**50.0**	41.7
`multi_step_ops`	66.7	66.7	**75.0**	25.0	**41.7**	25.0

Table 4: Macro-average VTSR (%) by category and harness ($n = 12$ per cell for both tiers). Li/Ba/St = Light/Balanced/Strict. **Bold** = best per tier×category.

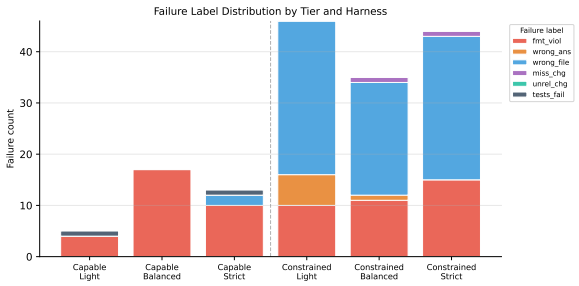

Figure 3: Failure label distribution by tier and harness condition. Capable models (top) are dominated by `format_violation`; constrained models (bottom) by `wrong_file`.

balanced (66.7% each) for frontier/strong models. For constrained models, balanced harness performs best or ties best in five of six categories, with strict generally performing no better than light. This pattern contrasts sharply with frontier chat models (where light dominates) and reasoning models (where strict dominates), consistent with the balanced harness providing just enough structural guidance without overloading constrained models' instruction-following capacity. The `multi_step_ops` category is universally the hardest for constrained models (25% light, 42% balanced, 25% strict), reflecting the inherent difficulty of coordinating multiple file operations under limited capacity.

4.7 Failure Label Distribution

Table 5 reports the failure label counts by model and harness.

`format_violation` is overwhelmingly the dominant failure mode introduced by complex harnesses in capable models: 25 of 26 failures in the balanced and strict conditions across Gemini 2.5 Flash and Qwen3.5-122B are format violations; the single exception is one `tests_still_fail` in Qwen3.5-122B strict, where the model understood the format but produced incorrect code

Model	**H**	**fv**	**wa**	**wf**	**mc**	**uc**	**tsf**
Gemini 2.5 Flash	Li	1	0	0	0	0	0
	Ba	10	0	0	0	0	0
	St	8	0	0	0	0	0
Qwen3.5-122B	Li	2	0	0	0	0	1
	Ba	6	0	0	0	0	0
	St	1	0	0	0	0	1
GPT-OSS-120B	Li	1	0	0	0	0	0
	Ba	1	0	0	0	0	0
	St	1	0	2	0	0	0
Qwen3.5:2B	Li	9	0	15	0	0	0
	Ba	2	0	7	1	0	0
	St	8	0	15	0	0	0
LLaMA 3.2	Li	0	5	15	0	0	0
	Ba	9	0	14	0	0	0
	St	6	0	12	1	0	0
Gemma4:e2B	Li	1	1	0	0	0	0
	Ba	0	1	1	0	0	0
	St	1	0	1	0	0	0

Table 5: Failure counts by model and harness. H = Harness (Li/Ba/St). fv = `format_violation`, wa = `wrong_answer`, wf = `wrong_file`, mc = `missing_change`, uc = `unrelated_change`, tsf = `tests_still_fail`. GPT-OSS-120B strict uses re-evaluated results (second API key). `unrelated_change` does not appear in any cell; it is retained in the taxonomy for completeness.

changes. No `wrong_answer` or `missing_change` failures appear in any balanced or strict cell for these models. This indicates that these models *understand* the tasks but fail to suppress explanatory prose when presented with process-heavy harness prompts. GPT-OSS-120B strict failures (1 `format_violation`, 2 `wrong_file`) are distributed across three tasks (T14, T16, T24) without a clear categorical pattern, consistent with model-level variance rather than systematic harness-induced failure. Constrained models (Qwen3.5:2B, LLaMA 3.2) exhibit a qualitatively different failure signature: `wrong_file` dominates under light harness, reflecting a tendency to act on the wrong file

without the structural guidance of an allowed-file list. LLaMA 3.2 alone shows five `wrong_answer` failures under light harness, the only tier×harness cell where this label appears prominently, suggesting limited task comprehension rather than format-compliance failure.

4.8 Inference Latency

Table 6 reports mean inference latency per task (seconds).

Model	Light	Balanced	Strict
Gemini 2.5 Flash	2.6	3.8	6.0
Qwen3.5-122B	35.4	38.5	**23.3**
GPT-OSS-120B	8.4	8.5	7.0
Qwen3.5:2B	15.2	8.5	15.2
LLaMA 3.2	1.3	2.7	2.9
Gemma4:e2B	16.0	21.2	22.2

Table 6: Mean inference latency (seconds per task). Gemini latency grows with harness complexity. Qwen3.5-122B shows *lower* latency under strict harness, consistent with explicit constraints reducing thinking-chain length. Constrained models (Ollama local) reflect hardware constraints; all values include model-load time on shared GPU.

Gemini's latency scales monotonically with harness complexity (2.6 s → 6.0 s), consistent with longer prompts generating more verbose outputs. Qwen3.5-122B shows the opposite trend: strict harness (23.3 s) is 34% faster than light (35.4 s), reinforcing the hypothesis that explicit constraints reduce the length of the model's internal thinking chain. GPT-OSS-120B latency is stable across conditions (7–9 s), suggesting it produces similarly sized outputs regardless of harness structure. Among constrained models, LLaMA 3.2 is the fastest (1.3–2.9 s) while Gemma4:e2B is the slowest (16–22 s); notably, Gemma4:e2B latency *increases* with harness complexity, mirroring Gemini's latency profile and consistent with its similar frontier-level output quality.

5 Discussion

Refuting the monotone inverse hypothesis. The central finding of this study is that harness sensitivity is *non-monotone* in capability tier. The monotone inverse hypothesis predicts a single gradient: as capability increases, optimal harness complexity decreases. Our results break this gradient in two places simultaneously. For the frontier *chat* model (Gemini 2.5 Flash), strict harness reduces VTSR by 29 pp—consistent with the hypothesis direction, but far larger in magnitude than expected. For the frontier *reasoning* model evaluated (Qwen3.5-122B), strict harness *increases* VTSR (+17 pp over balanced) and *reduces* latency —directly contradicting the hypothesis, which predicts that a high-capability model should need less harness structure. The balanced harness occupies an awkward middle ground that helps neither model type, suggesting that intermediate harness complexity is uniformly suboptimal.

These results refute the monotone framing and instead suggest that model *type* (chat vs. reasoning) is an independent moderating variable that capability tier alone cannot capture. We note, however, that the chat/reasoning distinction emerged as a *post-hoc* interpretive frame from observing the results; it was not a pre-specified moderating variable in the original study design. Future work should treat this as an exploratory finding requiring confirmatory replication with pre-registered hypotheses.

Tier-aware harness policy. The following guidelines reflect the specific model versions evaluated. Because model families evolve rapidly, more durable guidance targets model *type* (chat vs. reasoning) and instruction-tuning quality rather than specific model names; empirical re-evaluation is recommended whenever models are updated or replaced. Practitioners without benchmark access can approximate tier placement using a short harness probe (e.g., a 4–6 task subset of HEAT-24 covering `inspect_local` and `format_sensitive` categories), observing whether format violations concentrate under complex or simple conditions. Based on our results, we recommend:

- **Frontier-Proprietary (chat)**: Use light harness for JSON-output and format-sensitive tasks; reserve strict harness for file-editing tasks where format compliance is not at risk. Light harness is also cost-optimal, as shorter prompts reduce API token consumption. Avoid balanced harness, which combines the complexity of strict with less structured guidance.

- **Frontier-Reasoning (extended thinking)**: Use strict harness across all task categories. Explicit success criteria and verification specifications align with the model's chain-of-thought and reduce both error rate and latency.

- **Strong-Open**: Light and balanced harnesses are equally effective (95.8% each); strict harness incurs a modest -8.3 pp penalty (87.5%). For latency-sensitive deployments, light is preferred; for tasks requiring explicit constraints, strict remains viable.

- **Constrained (capable, e.g. Gemma4:e2B)**: Any harness works. This model's instruction-tuning quality renders it operationally equivalent to the strong-open tier; tier-aware routing based on parameter count alone would misclassify it.

- **Constrained (moderate, e.g. Qwen3.5:2B)**: Use balanced harness. The four-step process template and allowed-file list provide enough structural guidance to activate task understanding without overloading instruction-following capacity. Avoid light harness (catastrophic `wrong_file` failures) and strict harness (instruction overload collapses performance).

- **Constrained (low capability, e.g. LLaMA 3.2)**: No harness condition yields reliable performance; deployment of this tier for workspace-editing tasks is not recommended without further capability improvement.

Instruction-tuning quality supersedes parameter count. Gemma4:e2B achieves 91.7% VTSR at all harness conditions despite having ≈2 B parameters, matching the strong-open tier model (GPT-OSS-120B, 120 B parameters) on stability and approaching the frontier chat model (Gemini 2.5 Flash) on peak performance. This result challenges the common practice of classifying models into capability tiers by parameter count alone. For harness-sensitivity prediction, a model's instruction-tuning quality—its trained ability to comply with structured task directives and emit formatted outputs—is a more reliable predictor than raw model scale. Future tier-aware deployment frameworks should assess instruction-following capability empirically (e.g., on a held-out harness probe) rather than relying on parameter count as a proxy. We also note an alternative interpretation: Gemma4:e2B's result may indicate a tier-*taxonomy* failure rather than a capability insight. If this model genuinely belongs to the strong-open tier by instruction-tuning quality, its placement in the constrained tier may inflate apparent between-tier differences rather than demonstrating that parameter count is a poor proxy. Distinguishing these interpretations requires additional probing beyond HEAT-24.

Format violations as systemic risk. Across all capable models, `format_violation` is the dominant harness-induced failure, never `wrong_answer`. This is a critical observation: the models are capable of solving the tasks but fail operationally because they cannot resist injecting explanatory prose. Future harness designs should separate the process-instruction component from the output-format specification, presenting the latter immediately before the model's generation window (Deng et al., 2025). Alternatively, post-generation extraction (regex or schema-constrained decoding) could recover JSON fields from verbose outputs.

Reasoning models and thinking-mode interaction. The strict harness interacts with extended thinking modes in a way consistent with explicit constraints narrowing the reasoning model's search space, producing shorter thinking chains and better outputs. Whether this effect generalizes to other reasoning models or task domains remains an open question requiring controlled study.

Limitations and threats to validity. *External validity.* Our workspace is synthetic; real-world repositories are larger, noisier, and involve multi-file dependencies not present in our 12-file setup. Results may not transfer directly to production software engineering tasks such as those in SWE-bench (Jimenez et al., 2024).

Statistical. The experiment uses a single repeat per condition ($k = 1$); Wilson 95% confidence intervals for 24-task cells are wide (e.g., $58.3\% \rightarrow [36.6, 78.2]\%$), so individual cells should be interpreted as preliminary evidence. Future work should add $k \geq 3$ repetitions.

Internal validity. GPT-OSS-120B strict T13–T24 required re-evaluation with a second API key after the original run hit Groq's 200 k TPD rate limit; the re-run showed T14 and T16 (`wrong_file`) and T24 (`format_violation`) failing, which may reflect model variance rather than harness effects. Qwen3.5-122B was evaluated with extended thinking enabled across all harness conditions; we cannot isolate the contribution of extended thinking from the harness condition itself, so the non-monotonic pattern may reflect a thinking-mode–harness interaction rather than a

pure harness effect. Constrained-tier models run with 4096-token context Modelfiles; tasks requiring long outputs or large workspace context may be artificially penalised relative to unconstrained inference, potentially contributing to the low VTSR observed under some harness conditions.

Construct validity. Tier assignments are based on deployment characteristics (parameter count and infrastructure) to avoid circularity; the finding that Gemma4:e2B exceeds its assigned tier in performance is therefore a genuine empirical result, not an artifact of classification. Qwen3.5-122B is a Mixture-of-Experts model with ≈10B active parameters; its "Frontier-Reasoning" classification reflects extended-thinking capability, not parameter count.

6 Conclusion

The monotone inverse hypothesis—that higher-capability models need less harness structure, forming a predictable gradient—does not hold. Across 432 runs on HEAT-24, evidence suggests that harness sensitivity is non-monotone across the models evaluated, and depends jointly on model type (chat vs. reasoning) and instruction-tuning quality rather than capability tier alone: the evaluated frontier chat model benefits from light harness, the evaluated frontier reasoning model benefits from strict harness, and a 2 B constrained model matches strong-open stability regardless of harness condition. These results reject a single universal harness policy and call for empirical tier-aware and type-aware harness selection. HEAT-24, benchmark code, and full results will be released upon acceptance.

Acknowledgements

Experiments were conducted using the Google AI Studio API, Groq Cloud API (Groq, Inc.), a self-hosted vLLM inference service, and local Ollama inference. We thank the providers of open-weight models evaluated in this work.

References

Haikang Deng, Po-Nien Kung, and Nanyun Peng. 2025. Decoupling task-solving and output formatting in LLM generation. *arXiv preprint arXiv:2510.03595*.

Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. 2025. JSONSchemaBench: A rigorous benchmark of structured outputs for language models. *arXiv preprint arXiv:2501.10868*.

MD Azizul Hakim. 2026. Brevity constraints reverse performance hierarchies in language models. *arXiv preprint arXiv:2604.00025*.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-World GitHub issues? In *International Conference on Learning Representations*.

Imran Khan. 2025. You don't need prompt engineering anymore: The prompting inversion. *arXiv preprint arXiv:2510.22251*.

Yin Li. 2025. Decomposing LLM self-correction: The accuracy-correction paradox and error depth hypothesis. *arXiv preprint arXiv:2601.00828*.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, and 1 others. 2024. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*.

Renze Lou, Kai Zhang, and Wenpeng Yin. 2024. Large language model instruction following: A survey of progresses and challenges. *Computational Linguistics*.

Moran Mizrahi, Guy Kaplan, Dan Malkin, Rotem Dror, Dafna Shahaf, and Gabriel Stanovsky. 2024. State of what art? A call for multi-prompt LLM evaluation. *Transactions of the Association for Computational Linguistics*.

Sander Schulhoff, Michael Ilie, Nishant Balepur, and 1 others. 2024. The prompt report: A systematic survey of prompt engineering techniques. *arXiv preprint arXiv:2406.06608*.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2024. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations*.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.